Day 3 – Motion-Triggered Intelligence (With Code Integration)

---

1. Project Evolution and Status Overview

The Vision Guard project has successfully transitioned from its initial Proof of Concept (PoC) phase—characterized by a naive, continuous face-detection model—to a robust Minimum Viable Product (MVP) featuring a multi-modal trigger architecture. Moving beyond the baseline prototype established in Days 1–2, the "Day 3" iteration implements an event-driven surveillance logic. This evolution shifts the system from a simple reactive script to an intelligent, conditional alerting solution. By moving toward a "smart CCTV" framework, we have optimized the system to ignore environmental noise and only initiate high-level processing when specific movement criteria are met.

---

2. Motion Detection Technical Pipeline

The integrated motion detection sub-system employs a sophisticated foreground/background separation pipeline to quantify activity before engaging secondary classifiers. This sequential process is executed as follows:

Preprocessing

```
prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
curr_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

prev_gray = cv2.GaussianBlur(prev_gray, (9, 9), 0)
curr_gray = cv2.GaussianBlur(curr_gray, (9, 9), 0)
```

To minimize computational overhead, incoming frames are converted to a single-channel grayscale. A Gaussian Blur is applied using a (9,9) kernel to suppress high-frequency noise and sensor jitter, ensuring that subsequent analysis is performed on a stabilized image.

Temporal Differencing

```
diff = cv2.absdiff(prev_gray, curr_gray)
```

The system utilizes frame differencing to compare the current frame against a buffered previous frame. This identifies pixel-level changes over time, effectively isolating movement relative to the static background.

Thresholding & Morphology

```
_, thresh = cv2.threshold(diff, 40, 255, cv2.THRESH_BINARY)
thresh = cv2.dilate(thresh, None, iterations=2)
```

A binary threshold is applied to create a mask of moving pixels. Dilation expands the foreground regions, ensuring that moving subjects are treated as cohesive blobs rather than fragmented noise.

Quantification

```
motion_score = cv2.countNonZero(thresh)

if motion_score > 7000:
    motion_detected = True
else:
    motion_detected = False
```

Using pixel counting, the system quantifies movement intensity. An activation threshold of 7,000 pixels is required to validate movement as a significant event.

Frame Memory Update

```
prev_frame = frame.copy()
```

This ensures continuous temporal comparison between consecutive frames.

---

3. Integrated Decision Logic: The 'AND' Condition

The core architectural advancement in this version is the implementation of a logical "AND" gate for alert triggers.

Face Detection

```
faces = face_cascade.detectMultiScale(
    curr_gray,
    scaleFactor=1.2,
    minNeighbors=4,
    minSize=(60, 60)
)
```

Combined Logic

```
if motion_detected and len(faces) > 0:
    status = "ALERT"
else:
    status = "SAFE"
```

An 'ALERT' status now requires:

- Motion Detection Trigger (≥ 7,000 pixels)
- Facial Identification Trigger (Haar Cascade detection)

This multi-modal approach significantly optimizes CPU utilization and reduces false positives.

---

4. UI and Telemetry Enhancements

Face Count Display

```
cv2.putText(frame, f"Faces: {len(faces)}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
```

System Status Display

```
color = (0, 0, 255) if status == "ALERT" else (0, 255, 0)

cv2.putText(frame, f"Status: {status}", (10, 70),
            cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
```

Motion Telemetry

```
cv2.putText(frame, f"Motion: {motion_detected}", (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
```

Visual Overlays (Bounding Box + Label)

```
for (x, y, w, h) in faces:
    cv2.putText(frame, "Intruder", (x, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

    cv2.rectangle(frame, (x, y), (x + w, y + h),
                  (0, 255, 0), 2)
```

---

5. Updated System Workflow

The VisionGuard workflow has been synthesized to include both the original detection protocols and the new motion-based logic:

Input

Capture video frames:

```
ret, frame = cap.read()
```

Processing

Motion detection pipeline (as described above)

Logic

AND condition evaluation:

```
if motion_detected and len(faces) > 0:
```

Action

```
if motion_detected and len(faces) > 0:
    current_time = time.time()

    if current_time - last_capture_time > cooldown:
        print("Intruder Detected!")

        winsound.Beep(1000, 500)

        filename = f"captures/intruder_{image_count}.jpg"
        cv2.imwrite(filename, frame)

        image_count += 1
        last_capture_time = current_time
```

Control Mechanism (Cooldown)

Ensures system efficiency by limiting repeated triggers.

---

6. Development Conclusion and Next Steps

The Day 3 integration effectively mitigates the primary limitations identified in earlier builds, such as sensitivity to static noise and environmental lighting shifts. By utilizing motion as a prerequisite for face detection, the system is now both more accurate and computationally efficient.

Next Development Roadmap

- Hardware Integration (Arduino / ESP32)
- Remote Alerting & Cloud Storage
- Advanced Recognition (Authorized vs Unauthorized)